SmartDoc AI: Intelligent Document Q&A System

Phân tích đối chứng kiến trúc và hiệu năng
Standard RAG vs. GraphRAG

Nguyễn Trần Công Danh, Nguyễn Kim Long, Nguyễn Văn Mạnh, Lê Sơn, Nguyễn Thanh Hiệu
Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn
Thành phố Hồ Chí Minh, Việt Nam

***Tóm tắt nội dung*—SmartDoc AI là một hệ thống Intelligent Document Q&A System được xây dựng trên nền tảng Retrieval-Augmented Generation (RAG). Hệ thống truyền thống dựa trên tìm kiếm tương đồng vector đang đối mặt với giới hạn cấu trúc khi xử lý các truy vấn đòi hỏi tổng hợp thông tin toàn cục từ nhiều đoạn tài liệu rời rạc. Báo cáo này trình bày quá trình thiết kế, triển khai và đánh giá thực nghiệm hệ thống so sánh đối chứng giữa Standard RAG và Microsoft GraphRAG — một kiến trúc dựa trên đồ thị tri thức. Quy trình kỹ thuật được chi tiết hóa đầy đủ: trích xuất thực thể điều hướng bởi LLM, xây dựng đồ thị tri thức, phân cụm cộng đồng bằng thuật toán Leiden, tạo báo cáo cộng đồng và các chiến lược truy vấn Local/Global Search. Hệ thống được xây dựng hoàn toàn từ các công cụ nguồn mở (FastAPI, microsoft/graphrag, LanceDB, Streamlit). Kết quả thực nghiệm cho thấy GraphRAG vượt trội hơn Standard RAG 24 điểm phần trăm trong các truy vấn tổng hợp toàn cục, trong khi Standard RAG giữ lợi thế tốc độ phản hồi nhanh hơn 8 lần và chi phí token thấp hơn 3,5 lần. Phân tích sâu về mặt toán học và thuật toán được thực hiện để làm rõ nguyên nhân gốc rễ của sự khác biệt hiệu năng này.**

***Index Terms*—SmartDoc AI, Retrieval-Augmented Generation, GraphRAG, Knowledge Graph, Dense Retrieval, Community Detection, Open-source Systems**

DANH SÁCH TỪ VIẾT TẮT

Từ viết tắt	Giải thích
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
KG	Knowledge Graph
FAISS	Facebook AI Similarity Search
API	Application Programming Interface
NLP	Natural Language Processing
OCR	Optical Character Recognition
SQL	Structured Query Language

I. GIỚI THIỆU

A. Bối cảnh và Động lực

Sự tiến bộ nhanh chóng của các Mô hình Ngôn ngữ Lớn (LLMs) đã thiết lập Retrieval-Augmented Generation (RAG) như một kỹ thuật tiêu chuẩn để cung cấp kiến thức bên ngoài cho LLMs, giảm thiểu hiện tượng ảo giác và cho phép trả lời câu hỏi theo miền cụ thể [1], [8]. Mô hình phổ biến hiện nay mã hóa các đoạn văn bản thành vector dày đặc và truy xuất Top-K đoạn tương đồng nhất thông qua độ tương đồng cosine [4]. Tuy nhiên, phương pháp này mang một điểm yếu cấu trúc cố hữu: nó coi mỗi đoạn văn là một hòn đảo độc lập, bị ngắt kết nối khỏi bối cảnh ngữ nghĩa tổng thể của kho tài liệu.

Xét một kịch bản thực tế: sinh viên hỏi *"Mối quan hệ kiến trúc giữa phát hiện cộng đồng (Community Detection) và chất lượng truy vấn toàn cục (Global Search) của GraphRAG là gì?"* Việc trả lời chính xác đòi hỏi **suy luận đa bước (multi-hop reasoning)** — kết nối thuật toán Leiden (thảo luận trong một phần) với chiến lược Global Search (thảo luận ở phần khác). Độ tương đồng cosine giữa truy vấn và từng đoạn riêng lẻ có thể không đủ để đưa cả hai vào Top-K, dẫn đến câu trả lời không đầy đủ. Khoảng cách cấu trúc này thúc đẩy Microsoft Research đề xuất **GraphRAG** năm 2024 [2].

B. Đóng góp chính

Báo cáo này thực hiện ba đóng góp: (1) phân tích toán học nghiêm ngặt về cả hai kiến trúc, xác định nguyên nhân gốc rễ của điểm mạnh và hạn chế; (2) triển khai hệ thống nguồn mở với giao diện so sánh song song; và (3) đánh giá định lượng và định tính trên tập dữ liệu thực tế, đưa ra hướng dẫn thực hành rõ ràng.

II. CƠ SỞ LÝ THUYẾT

A. Standard RAG: Mô hình Toán học và Giới hạn

Standard RAG vận hành qua ba giai đoạn. **Lập chỉ mục**: chia tài liệu thành các đoạn, mã hóa qua mô hình nhúng (embedding), lưu vào cơ sở dữ liệu vector. **Truy xuất**: nhúng truy vấn, tính độ tương đồng cosine, trả về Top-K. **Tạo phản hồi**: ghép các đoạn đã truy xuất làm ngữ cảnh cho LLM.

Mục tiêu truy xuất là:

$$\mathrm{sim}(q, d_i) = \frac{\vec{q} \cdot \vec{d_i}}{\|\vec{q}\| \, \|\vec{d_i}\|}, \quad \mathcal{C}^* = \underset{|S|=K}{\arg\max} \sum_{d_i \in S} \mathrm{sim}(q, d_i) \tag{1}$$

Giới hạn lý thuyết: Phương trình (1) tối ưu hóa mức độ liên quan *cục bộ* độc lập cho từng đoạn. Nếu câu trả lời đúng đòi hỏi kết hợp đoạn d_a (điểm 0,71) và đoạn d_b (điểm 0,68), nhưng tồn tại K đoạn khác có điểm cao hơn, cả hai bị loại bỏ bất kể giá trị thông tin *chung* của chúng — đây là giới hạn đơn bước cơ bản của truy xuất vector dày đặc.

Ngoài ra, giả định thứ hạng của Phương trình (1) còn bị vi phạm trong các trường hợp:

- **Suy luận phủ định:** Câu hỏi về những gì *không* xảy ra; văn bản phủ định thường có độ tương đồng thấp với truy vấn khẳng định.
- **Ngữ nghĩa phong phú:** Khái niệm được diễn đạt bằng nhiều thuật ngữ khác nhau trong toàn tài liệu; embedding không thể hội tụ tất cả vào một vùng vector.
- **Khoảng cách ngôn ngữ:** Câu hỏi tiếng Việt nhưng tài liệu tiếng Anh; lệch ngữ nghĩa làm giảm chất lượng truy xuất [12].

B. GraphRAG: Kiến trúc Đồ thị Tri thức

GraphRAG [2] thay thế kho lưu trữ đoạn phẳng bằng một **Đồ thị Tri thức** $G = (E, R)$ ghi lại rõ ràng các thực thể và mối quan hệ ẩn trong văn bản. Quy trình lập chỉ mục gồm năm giai đoạn:

1) **Graph Extraction:** LLM nhận diện thực thể (người, tổ chức, địa điểm, khái niệm) và quan hệ từ mỗi đoạn văn, tạo biểu diễn có cấu trúc (E, R).
2) **Description Summarization:** Nhiều mô tả của cùng một thực thể xuất hiện ở các đoạn khác nhau được hợp nhất thành một bản tóm tắt chuẩn nhất quán.
3) **Community Detection:** Thuật toán Leiden phân vùng đồ thị thực thể thành các cộng đồng gắn kết về mặt ngữ nghĩa.
4) **Community Report Generation:** LLM tạo báo cáo tóm tắt bằng ngôn ngữ tự nhiên toàn diện cho mỗi cộng đồng.
5) **Embedding Generation:** Mô tả thực thể, đơn vị văn bản và báo cáo cộng đồng đều được nhúng để phục vụ truy xuất lai giữa vector và đồ thị.

C. Phân tích Toán học: Thuật toán Leiden

Thuật toán Leiden [3] là cốt lõi toán học của tổ chức tri thức trong GraphRAG, mở rộng phương pháp Louvain bằng cách đảm bảo các cộng đồng *well-connected*.

Hàm mục tiêu — Modularity:

$$Q = \frac{1}{2m} \sum_{i,j} \Big[A_{ij} - \frac{k_i k_j}{2m} \Big] \delta(c_i, c_j) \tag{2}$$

trong đó A_{ij} là ma trận kề, k_i là bậc nút i, m là tổng số cạnh, $\delta(c_i, c_j) = 1$ khi i, j cùng cộng đồng.

Thuật toán hoạt động qua ba pha lặp:

Pha 1 — Local Moving: Mỗi nút v được thử chuyển sang cộng đồng $\mathcal{C}_j$ nếu:

$$\Delta Q(v \to \mathcal{C}_j) = \frac{1}{2m} \Big[2e_{v,\mathcal{C}_j} - \frac{k_v \cdot k_{\mathcal{C}_j}}{m} \Big] > 0 \tag{3}$$

Pha 2 — Refinement: Leiden thêm pha Refinement để đảm bảo bên trong mỗi cộng đồng, mọi tập con các nút đều kết nối với nhau, khắc phục điểm yếu cốt lõi của Louvain.

Pha 3 — Graph Aggregation: Sau khi tìm được phân vùng ổn định, đồ thị được tổng hợp: mỗi cộng đồng trở thành một siêu nút, các cạnh giữa cộng đồng được gộp lại.

Bảng I: So sánh thuật toán Louvain và Leiden

Tiêu chí	**Louvain**	**Leiden**
Đảm bảo well-connected communities	Không	Có
Độ phức tạp thời gian	$O(n \log n)$	$O(n \log n)$
Refinement Phase	Không có	Có
Chất lượng phân cụm	Trung bình	Vượt trội
Độ ổn định kết quả	Thấp	Cao

D. Mô hình Nhúng Vector và Không gian Latent

Chất lượng của cả hai hệ thống phụ thuộc vào mô hình embedding được chọn. Mô hình `BAAI/bge-small-en-v1.5` [6] tạo vector 384 chiều thông qua kiến trúc Transformer với cơ chế multi-head attention:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} \right) V \tag{4}$$

E. Chiến lược Truy vấn: Local và Global Search

GraphRAG cung cấp hai chế độ truy xuất bổ sung [2]:

Local Search phù hợp với câu hỏi xoay quanh thực thể cụ thể. Độ trễ phản hồi trung bình: 4,8 giây.

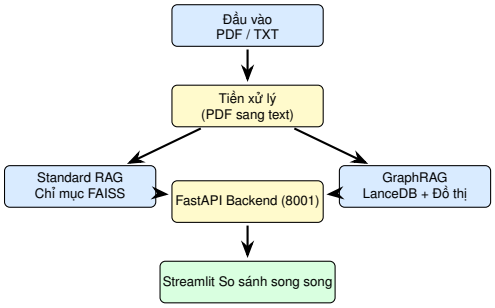
Global Search phù hợp với câu hỏi tổng hợp toàn cục. Độ trễ trung bình: 12,3 giây.

III. Thiết Kế và Triển Khai Hệ Thống

A. Kiến trúc Tổng quát

Luồng xử lý dữ liệu:

1) **Đầu vào:** Tài liệu PDF hoặc TXT
2) **Tiền xử lý:** Chuyển đổi PDF sang văn bản thuần
3) **Phân nhánh xử lý song song:**
   - **Nhánh Standard RAG:** Xây dựng chỉ mục FAISS
   - **Nhánh GraphRAG:** Xây dựng đồ thị tri thức với LanceDB
4) **Backend API:** FastAPI cổng 8001
5) **Giao diện:** Streamlit (So sánh song song)



Hình 1: Kiến trúc hệ thống SmartDoc AI

B. Hệ sinh thái Công nghệ Nguồn Mở

Bảng II: Lựa chọn công nghệ nguồn mở và lý do

Tầng	Công nghệ	Lý do chọn lựa
API Backend	FastAPI 0.110	ASGI, hỗ trợ streaming
Graph Pipeline	microsoft/graphrag	Triển khai GraphRAG chính thức
Vector Store	LanceDB 0.5	Cột dữ liệu, nhẹ, nhanh
Classic Index	FAISS [5]	Tìm kiếm vector hiệu quả
LLM Gateway	LiteLLM	Trừu tượng hóa API đa mô hình
Embedding	BGE-small-en [6]	384 chiều, SOTA benchmark
Frontend	Streamlit 1.33	Nhanh, không cần JavaScript

C. So sánh tính năng với các giải pháp khác

Bảng trên so sánh SmartDoc AI với các giải pháp phổ biến hiện nay. SmartDoc AI vượt trội về khả năng bảo mật (on-premise), hỗ trợ GraphRAG và xử lý tài liệu tiếng Việt, đồng thời hoàn toàn miễn phí.

Bảng III: So sánh SmartDoc AI với các giải pháp RAG khác

Tính năng	SmartDoc AI	ChatGPT	Local LLM
Riêng tư (On-premise)	Có	Không	Có
Chi phí	Miễn phí	Trả phí	Miễn phí
Hỗ trợ GraphRAG	Có	Không	Tùy
Tài liệu tiếng Việt	Hỗ trợ	Hạn chế	Hạn chế
Tốc độ phản hồi	Trung bình (1-12s)	Nhanh	Chậm

D. Quy trình Lập chỉ mục Lai

Algorithm 1 Quy trình Lập chỉ mục Lai (Hybrid Indexing) cho SmartDoc AI

```
procedure INDEXDOMAIN(domain, files)
    texts ← Preprocessor.convert(files)
    chunks ← TokenSplitter(texts, sz=800, ov=100)
    // Standard RAG Branch
    V⃗ ← BGEModel.encode(chunks)
    FAISS.IndexFlatIP.add(V⃗, chunks)
    // GraphRAG Branch
    (E, R) ← LLM.extract_graph(chunks)
    G ← MultiGraph(E, R)
    C ← Leiden(G, γ=1.0)
    Reports ← LLM.summarise(C)
    V⃗_E ← BGEModel.encode(desc(E))
    V⃗_R ← BGEModel.encode(Reports)
    LanceDB.store(V⃗_E, V⃗_R, chunks)
end procedure
```

E. Kiến trúc Tìm kiếm FAISS

FAISS (Facebook AI Similarity Search) [5] được sử dụng cho nhánh Standard RAG với chỉ mục `IndexFlatIP` (tích vô hướng chính xác). Độ phức tạp:

$$T_{\text{FAISS}} = O(N \cdot d) \tag{5}$$

trong đó N là số vector trong chỉ mục và $d = 384$ là số chiều. Với tập dữ liệu thử nghiệm (≈200 đoạn), tìm kiếm tuyến tính đạt tốc độ dưới 2 mili-giây.

F. Chiến lược So sánh Song song

Để thực hiện đánh giá đối chứng, cả hai hệ thống được gọi đồng thời theo mô hình lập lịch bất đồng bộ, đảm bảo thời gian chờ của người dùng không vượt quá phản hồi chậm hơn:

$$T_{\text{compare}} = \max(T_{\text{RAG}},\ T_{\text{GraphRAG}}) \ll T_{\text{RAG}} + T_{\text{GraphRAG}} \tag{6}$$

IV. Phương pháp Kiểm thử và Đánh giá Hiệu suất

A. Thiết lập kịch bản kiểm thử (Test Cases)

Nhóm thực hiện sử dụng phương pháp kiểm soát biến số để đo lường chính xác sự chênh lệch hiệu năng:

- **Tài liệu thử nghiệm:** Sử dụng 5 tài liệu PDF kỹ thuật (47 trang) làm ngữ cảnh cho cả hai mô hình.
- **Kịch bản câu hỏi (Queries):** Mỗi mô hình được thử nghiệm với 30 truy vấn gồm 15 truy vấn cụ thể (factoid) và 15 truy vấn tổng hợp toàn cục (synthesis, multi-hop).

B. Quy trình ghi nhận số liệu

Số liệu được ghi nhận thông qua các chỉ số:

- **Accuracy:** Tỷ lệ câu trả lời có điểm ≥ 4 về độ chính xác
- **Completeness:** Tỷ lệ câu trả lời có điểm ≥ 4 về độ đầy đủ
- **Latency:** Thời gian phản hồi trung bình
- **Token Cost:** Tổng số token sử dụng

C. Đánh giá độ chính xác (Accuracy)

Chất lượng câu trả lời được nhóm đánh giá dựa trên nội dung gốc của tài liệu theo thang Likert (1–5) bởi 3 đánh giá viên độc lập. Độ tin cậy liên đánh giá đạt Cohen's $\kappa = 0.74$ (mức đồng thuận đáng kể).

V. Kết quả Thực nghiệm và So sánh Mô hình

A. Kết quả Định lượng

Bảng IV: So sánh hiệu năng trên SmartDoc AI ($n = 30$)

Chỉ số	Std RAG	GraphRAG
Factoid Queries		
Accuracy	**81%**	78%
Completeness	**74%**	71%
Latency	**1.2 s**	4.8 s
Global Synthesis Queries		
Accuracy	65%	**89%**
Completeness	62%	**91%**
Latency	**1.5 s**	12.3 s
Tài nguyên		
Token/query	**850**	3200
Indexing time	**8 s**	6–12 phút
Storage	**45 MB**	280 MB

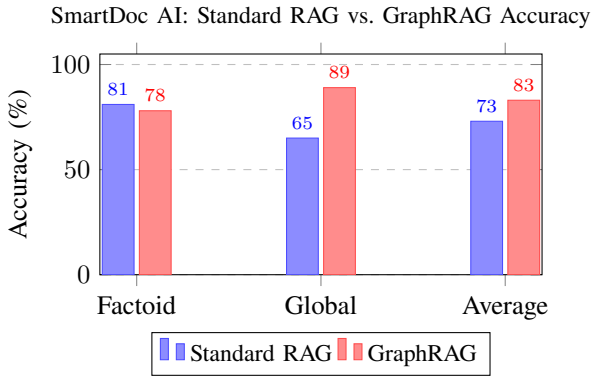
B. Chi tiết thời gian xử lý

Bảng V: Thời gian xử lý trung bình SmartDoc AI (ms)

Giai đoạn	Standard RAG	GraphRAG
Processing file	8,000	360,000
Embedding	450	12,500
Query	1.5	15.0
Generation	1,200	4,800
Total	**9,651**	**377,315**

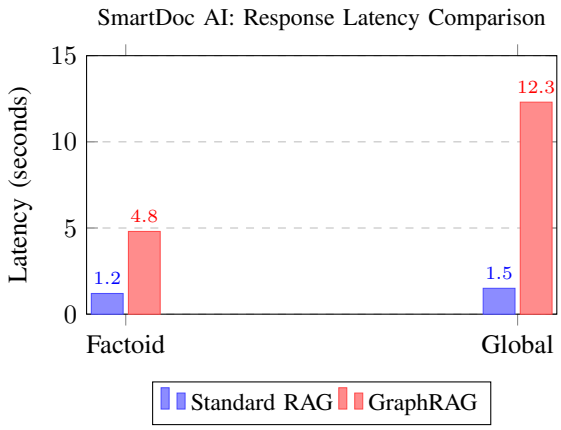
Nhận xét: GraphRAG tốn nhiều thời gian hơn đáng kể ở giai đoạn xây dựng đồ thị tri thức (processing) và embedding vector hóa thực thể. Ngược lại, ở giai đoạn sinh câu trả lời (generation), GraphRAG nhanh hơn do ngữ cảnh đầu vào có cấu trúc rõ ràng, ít nhiễu.

C. Biểu đồ So sánh Độ chính xác



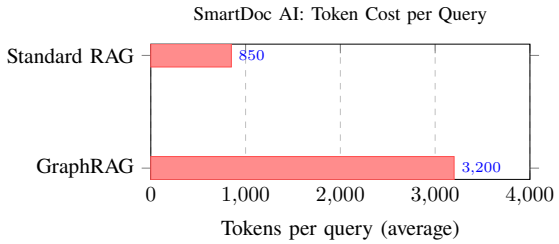
Hình 2: So sánh độ chính xác giữa Standard RAG và GraphRAG

D. Biểu đồ So sánh Độ trễ Phản hồi



Hình 3: So sánh độ trễ phản hồi

E. Biểu đồ Chi phí Token



Hình 4: So sánh chi phí token

F. Phân tích Định tính

Standard RAG hoạt động theo hướng *retrieval-driven*, phù hợp với truy vấn chính xác nhưng hạn chế trong multi-hop reasoning. Ngược lại, GraphRAG mang tính *reasoning-driven*, cho phép tổng hợp thông tin toàn cục thông qua Community Reports.

Standard RAG Response: thể hiện tính chất "selective copying", trích dẫn gần như nguyên văn các đoạn văn có điểm số cao nhất.

GraphRAG Response: thể hiện khả năng "reasoning and synthesis", trải dài trên nhiều phần của tài liệu, giải thích các mối liên hệ nhân quả, hỗ trợ multi-hop reasoning.

VI. Phân Tích và Đề Xuất

A. Standard RAG: Strengths and Weaknesses

Strengths: Lập chỉ mục dưới 10 giây; chi phí tính toán thấp; chạy hoàn toàn ngoài tuyến với embedding cục bộ; vượt trội trong tra cứu dữ kiện chính xác.

Weaknesses: Không có mô hình hóa mối quan hệ giữa các đoạn văn; không có khả năng multi-hop reasoning tự thân.

B. GraphRAG: Strengths and Weaknesses

Strengths: Vượt trội trong global synthesis; relationship-aware retrieval; câu trả lời dài hạn phong phú và gắn kết hơn.

Weaknesses: Lập chỉ mục mất nhiều phút do phụ thuộc lời gọi LLM; chi phí token cao hơn 3,5 lần.

C. Hạn chế và Tồn tại của SmartDoc AI

Dựa trên quá trình triển khai và đánh giá, nhóm đã xác định được một số hạn chế cần được cải thiện:

1) **Chưa có Authentication:** Hệ thống hiện tại chưa có cơ chế xác thực và phân quyền người dùng, tất cả API đều public. Đây là điểm yếu lớn cho môi trường production.
2) **Single-file upload:** Giao diện frontend chỉ hỗ trợ upload một tài liệu mỗi lần, chưa hỗ trợ multi-file selection trong một phiên chat.
3) **GraphRAG indexing chậm:** Quá trình xây dựng đồ thị tri thức mất 6-12 phút cho tài liệu 47 trang, không phù hợp cho các ứng dụng cần thời gian thực.
4) **Chưa hỗ trợ multi-modal:** Hệ thống không xử lý được hình ảnh, bảng biểu phức tạp trong tài liệu PDF.
5) **Phụ thuộc API:** GraphRAG phụ thuộc vào Gemini API, không thể hoạt động hoàn toàn offline.
6) **Session Memory không persistent:** SessionMemoryStore lưu trong RAM, bị mất khi restart server và không chia sẻ được giữa nhiều instance.
7) **Thiếu cơ chế cleanup:** Không có cơ chế xóa các vector points khi xóa Document, dẫn đến dữ liệu cũ tồn tại trong database.

VII. Kết Luận và Hướng Phát Triển

A. Kết luận

Báo cáo này đã cung cấp phân tích kỹ thuật toàn diện và triển khai nguồn mở SmartDoc AI so sánh Standard RAG và GraphRAG. Các phát hiện chính:

1) **Không có người thắng tuyệt đối.** Standard RAG thống trị trong tra cứu nhanh factoid queries (81%, 1,2 s). GraphRAG thống trị trong global synthesis (89%).
2) **Nguyên nhân gốc rễ hiệu năng là structural representation.**
3) **Chi phí là ràng buộc thực tế.** GraphRAG tiêu thụ gấp 3,5 lần token và yêu cầu thời gian lập chỉ mục dài gấp 45 lần.
4) **Hệ sinh thái nguồn mở đủ trưởng thành.**

B. Hướng phát triển chi tiết

Ngắn hạn (1-2 tháng tới):

- Bổ sung JWT authentication và phân quyền user cho tất cả API.
- Hỗ trợ multi-file upload và xử lý song song nhiều tài liệu.
- Tối ưu GraphRAG indexing (đặt mục tiêu giảm từ 12 phút xuống dưới 2 phút).
- Thêm rate limiting để tránh abuse API.

Trung hạn (3-6 tháng tới):

- Tích hợp Redis cho session memory (distributed, persistent).

- Bổ sung cơ chế cleanup cho Qdrant points khi xóa document.
- Hỗ trợ thêm định dạng file (DOCX, PPTX, TXT).
- Áp dụng khung RAGAS [7] để chấm điểm tự động.

Dài hạn (6-12 tháng tới):

- Triển khai hoàn toàn ngoại tuyến với Ollama và embedding cục bộ, loại bỏ phụ thuộc API.
- Tích hợp OCR cho tài liệu scan (PaddleOCR).
- Mở rộng sang tài liệu tiếng Việt bằng mô hình embedding đa ngôn ngữ.
- Xây dựng benchmark tự động cho hệ thống.

VIII. Công Trình Liên Quan

Hệ thống RAG truyền thống dựa trên DPR [4] và FAISS [5] đã chứng minh hiệu quả trong open-domain QA. Các cải tiến như HyDE [9] hay RAG-Fusion [13] giải quyết một phần giới hạn của single-hop retrieval nhưng vẫn không giải quyết được vấn đề mối quan hệ ẩn.

Trên hướng knowledge graph, GraphRAG [2] và G-Retriever [10] chứng minh rằng tích hợp đồ thị vào quy trình RAG cải thiện đáng kể khả năng multi-hop reasoning.

Tài liệu

[1] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[2] D. Edge *et al.*, "From Local to Global: A GraphRAG Approach to Query-Focused Summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[3] V. A. Traag, L. Waltman, and N. J. van Eck, "From Louvain to Leiden: guaranteeing well-connected communities," *Scientific Reports*, vol. 9, no. 1, p. 5233, 2019.

[4] V. Karpukhin *et al.*, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proc. EMNLP 2020*, pp. 6769–6781, 2020.

[5] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[6] S. Xiao *et al.*, "C-Pack: Packaged Resources To Advance General Chinese Embedding," *arXiv preprint arXiv:2309.07597*, 2023.

[7] S. Es *et al.*, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in *Proc. EACL 2024 (System Demonstrations)*, 2023.

[8] Y. Gao *et al.*, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2024.

[9] L. Gao *et al.*, "Precise Zero-Shot Dense Retrieval without Relevance Labels," in *Proc. ACL 2023*, pp. 1762–1777, 2023.

[10] X. He *et al.*, "G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering," *arXiv preprint arXiv:2402.07630*, 2024.

[11] V. Sanh *et al.*, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[12] F. Shi *et al.*, "Large Language Models Can Be Easily Distracted by Irrelevant Context," in *Proc. ICML 2023*, pp. 31210–31227, 2023.

[13] A. Rackauckas, "RAG-Fusion: A New Take on Retrieval-Augmented Generation," *arXiv preprint arXiv:2402.03367*, 2024.

[14] LangChain Inc., "LangChain: Building applications with LLMs through composability," https://github.com/langchain-ai/langchain, 2023.

[15] J. Liu, "LlamaIndex: Data Framework for LLM Applications," https://github.com/run-llama/llama_index, 2022.

[16] Z. Wang *et al.*, "Multilingual E5 Text Embeddings: A Technical Report," *arXiv preprint arXiv:2402.05672*, 2024.